Name: Samir Kumar Gupta Roll No.: 2301730243 Environment: Ubuntu 24.04.5 LTS (OrbStack VM, arm64)

Question 5 - Deploy an application to Kubernetes (Minikube)

Question 5: Deploy an application to a Kubernetes (Minikube) cluster using Deployment and Service manifests, and verify the running pods.

Executed on an Ubuntu 24.04.5 LTS OrbStack machine (`ensp461-lab2`, arm64) with Minikube v1.39.0 (docker driver), Kubernetes v1.37.0 and kubectl v1.37.2. Source, logs and screenshots: https://github.com/ksamirdev/ensp461-lab2.

Cluster and registry

```
$ minikube start --driver=docker --memory=2048 --cpus=2 --insecure-registry=lab2-registry:5000
\
    --base-image=docker.io/kicbase/stable:v0.0.51
* Preparing Kubernetes v1.37.0 on containerd 2.3.4 ...
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
$ docker network connect minikube lab2-registry
$ kubectl get nodes -o wide
NAME       STATUS     ROLES           VERSION   INTERNAL-IP    OS-IMAGE
minikube   NotReady   control-plane   v1.37.0   192.168.49.2   Debian GNU/Linux 12 (bookworm)
```

The image is not loaded into the node by hand. The registry from Q2 is attached to the `minikube` Docker network, and the node pulls `lab2-registry:5000/ensp461-lab2:<tag>` from it like a real cluster pulls from a registry. `--insecure-registry` allows plain HTTP to that registry, which is acceptable only because it never leaves the machine. The node was briefly `NotReady` right after start while the CNI came up; the Deployment waited for it.

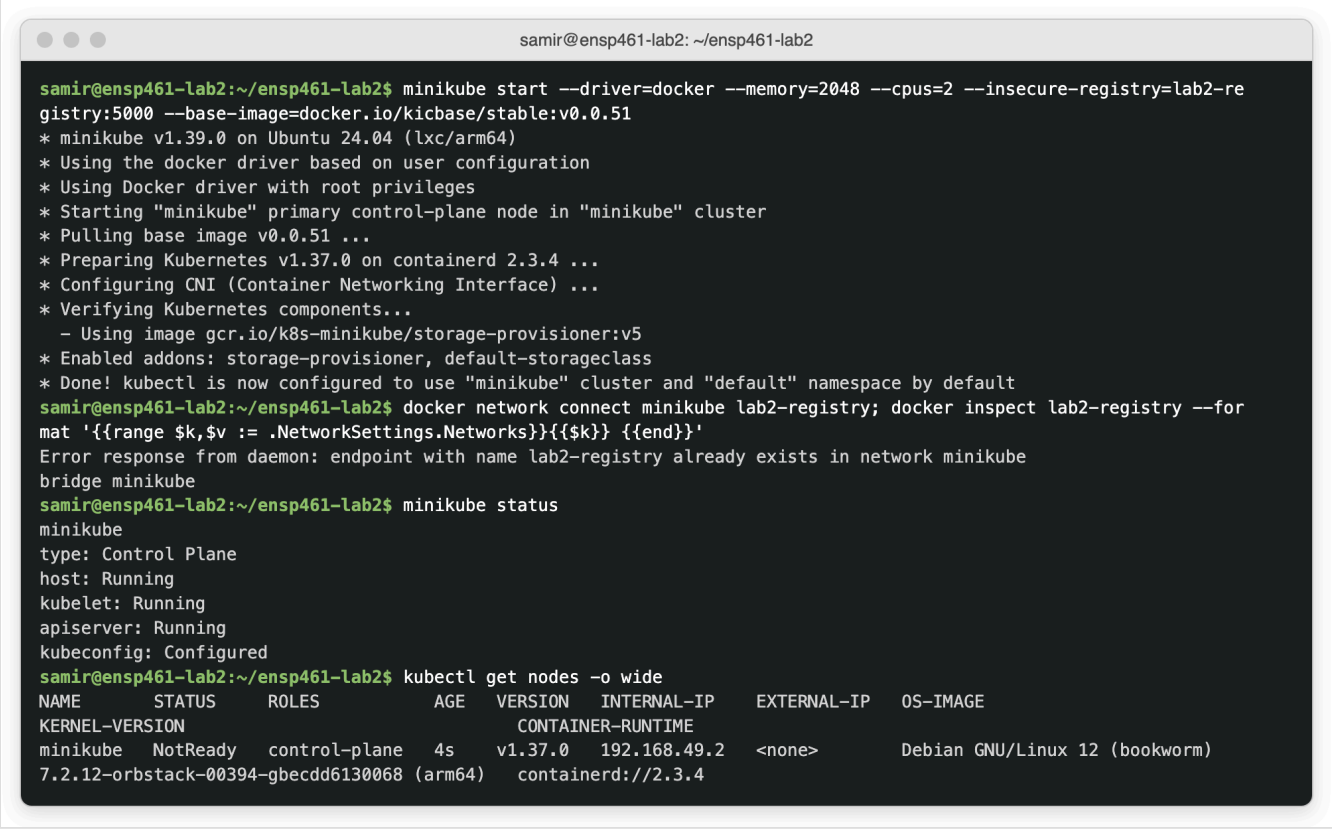


Fig: Minikube start, registry attached to the cluster network, node status

Manifests

`k8s/configmap.yaml`

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: lab2-config
data:
  APP_ENV: minikube
```

`k8s/deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab2-web
  labels: {app: lab2-web}
spec:
  replicas: 3
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate: {maxSurge: 1, maxUnavailable: 0}
  selector:
    matchLabels: {app: lab2-web}
  template:
    metadata:
      labels: {app: lab2-web}
    spec:
      containers:
        - name: web
          image: lab2-registry:5000/ensp461-lab2:2.0.0
          ports: [{containerPort: 5000}]
          envFrom: [{configMapRef: {name: lab2-config}}]
          resources:
            requests: {cpu: 50m, memory: 64Mi}
            limits: {memory: 192Mi}
          lifecycle:
            preStop: {exec: {command: ["sleep", "5"]}}
          readinessProbe: {httpGet: {path: /health, port: 5000}, periodSeconds: 3}
          livenessProbe: {httpGet: {path: /health, port: 5000}, initialDelaySeconds: 10}
```

`k8s/service.yaml`

```
apiVersion: v1
kind: Service
metadata:
  name: lab2-web
spec:
  type: NodePort
  selector: {app: lab2-web}
  ports:
    - {port: 80, targetPort: 5000, nodePort: 30080}
```

Object	Purpose
ConfigMap `lab2-config`	configuration outside the image (`APP_ENV`), injected as environment variables

Object	Purpose
Deployment `lab2-web`	3 replicas, rolling updates with no unavailable pods, probes, resource requests and limits
Service `lab2-web`	stable virtual IP and DNS name in front of the pods; NodePort 30080 for access from the machine

The readiness probe decides when a pod receives traffic, while the liveness probe restarts a pod that stops answering. The `preStop` sleep was added after Q6 measured failed requests during rollouts (see Q6).

Deploy and verify

```
$ kubectl apply -f k8s/
configmap/lab2-config created
deployment.apps/lab2-web created
service/lab2-web created
$ kubectl rollout status deployment/lab2-web --timeout=180s
deployment "lab2-web" successfully rolled out
$ kubectl get pods -l app=lab2-web -o wide
NAME                        READY   STATUS    RESTARTS   AGE    IP           NODE
lab2-web-fd84f8f84-57wfq   1/1     Running   0          114s   10.244.0.4   minikube
lab2-web-fd84f8f84-d9hs4   1/1     Running   0          114s   10.244.0.2   minikube
lab2-web-fd84f8f84-xqfrb   1/1     Running   0          114s   10.244.0.3   minikube
$ kubectl get service lab2-web
NAME       TYPE       CLUSTER-IP    PORT(S)
lab2-web   NodePort   10.99.43.65   80:30080/TCP
```

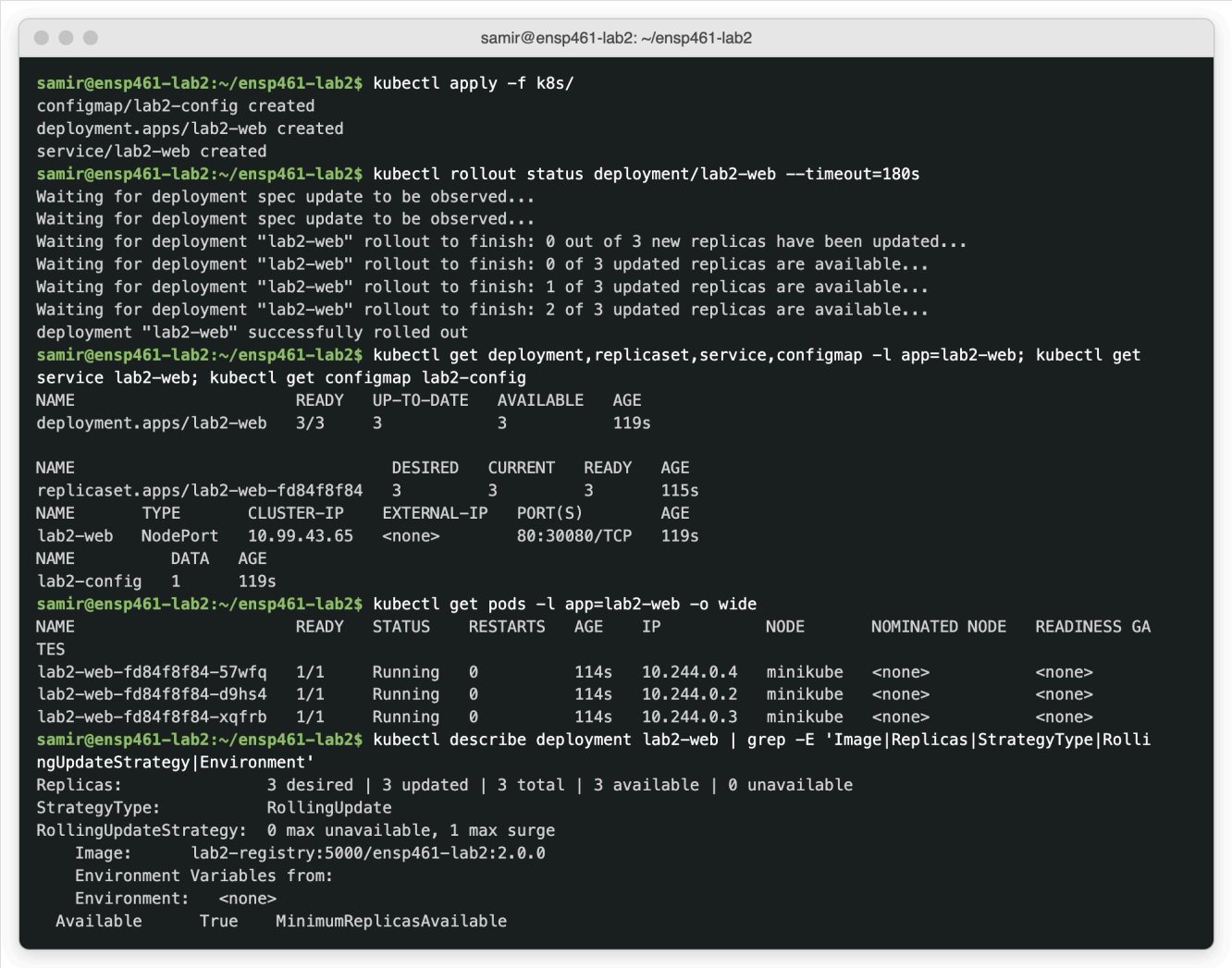


Fig: Apply, rollout status, Deployment, ReplicaSet, Service and pods

The Deployment created ReplicaSet `lab2-web-fd84f8f84`, which created three pods, each with its own pod IP on the `10.244.0.0/16` pod network.

Access through the Service

```
$ curl -s http://$(minikube ip):30080/
{"environment":"minikube","owner":"Samir Kumar Gupta","served_by":"lab2-web-fd84f8f84-xqfrb","s
ervice":"ensp461-lab2","version":"2.0.0"}
$ for i in 1 2 3 4 5 6; do curl -s http://$(minikube ip):30080/ | ... served_by; done
lab2-web-fd84f8f84-xqfrb     # 4 of 6 requests
lab2-web-fd84f8f84-d9hs4     # 2 of 6 requests
```

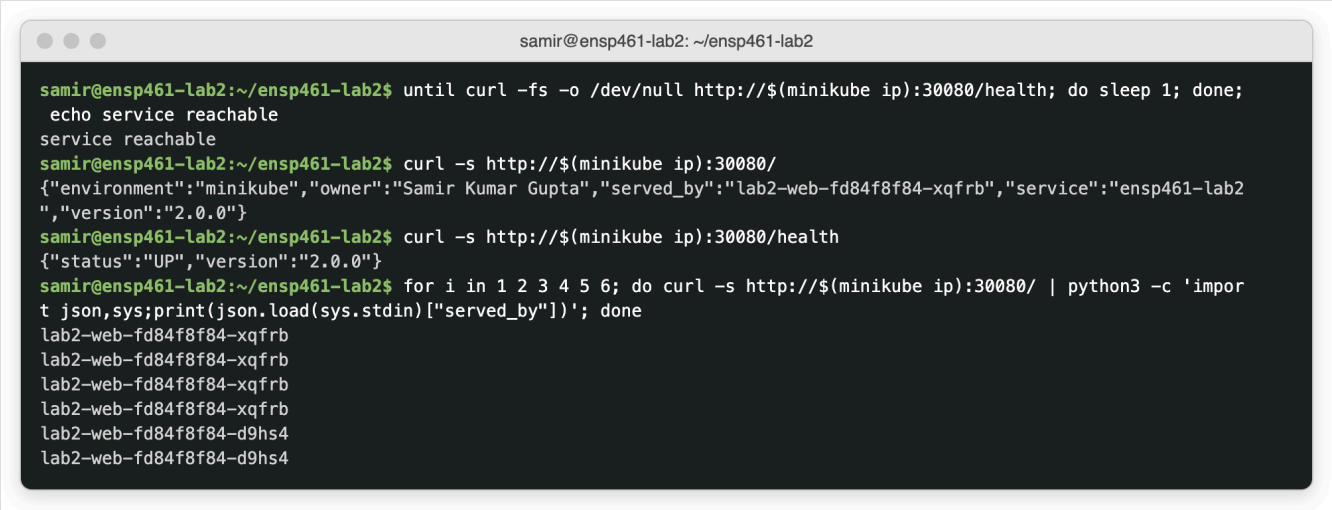


Fig: Requests through the NodePort, answered by different pods

Observations:

- `environment` is `minikube`, so the ConfigMap value reached the container.
- Requests to one NodePort were answered by different pods: kube-proxy spreads connections across the Service's ready endpoints.
- In the first deployment (`docs/logs/q5-k8s-run1.log`) the first `curl` came back empty even though the rollout had finished. kube-proxy had not yet programmed the NodePort rules. The script now waits until the Service answers before testing it.